

Finite-Domain Conflict-Free xApp Coordination: A Hybrid Continuous-Inference and Exact-Repair Framework for O-RAN

Lillia Ouali^{1*}, Madani Bezoui², Kamal Amroun¹, Ahcene Bounceur³

^{1*}Department of Computer Science, University of Bejaia, Bejaia, Algeria.

²LINEACT, CESI, Nancy, France.

³University of Sharjah, Sharjah, United Arab Emirates.

*Corresponding author(s). E-mail(s): lillia.ouali@univ-bejaia.dz;

Contributing authors: mbezoui@cesi.fr; kamal.amroun@univ-bejaia.dz;
abounceur@sharjah.ac.ae;

Abstract

The programmability of the Open Radio Access Network (O-RAN) allows independently developed extended applications, called xApps, to control radio access network parameters through the near-real-time RAN Intelligent Controller. However, concurrently deployed xApps may issue incompatible actions, compete for shared resources, or indirectly degrade each other’s key performance indicators. Existing conflict-management approaches primarily rely on static priorities, learned performance predictors, digital twins, or joint training. This paper investigates a complementary failure mode that arises when discrete xApp coordination is addressed through continuous relaxation. We formulate conflict-free xApp arbitration as a finite-domain constraint optimization problem and analyze non-integral fixed points of a relation-conditioned product-of-experts update. In the smallest arbitration motif we prove the existence of an exact fractional fixed point; in larger instances we detect its operational counterpart, a stable continuous configuration whose deterministic decoding still produces an invalid or service-level-agreement-violating control decision. We then introduce VERIXAPP, a verified hybrid framework that combines continuous marginal inference, explicit stall diagnostics, exact repair of a conflict-induced subproblem, and an independent decision verifier. The framework distinguishes direct, indirect, and implicit conflicts and returns a control action only after validating all registered hard constraints. The experimental protocol evaluates the approach on reproducible O-RAN scenarios involving energy saving, mobility robustness, load balancing, power control, and radio-resource allocation. Comparisons include uncoordinated execution, static-priority arbitration, continuous-only coordination, and complete constraint programming. The results show that continuous-only coordination stalls on approximately one quarter of decision epochs and reaches only 74.7% verified feasibility, whereas both complete CP-SAT and VERIXAPP attain 100% verified feasibility. On synthetic instances of this size the exact CP-SAT solver is itself fast and complete (median 6 ms), so the continuous stage of VERIXAPP adds latency (median 42 ms) rather than reducing it; the contribution of VERIXAPP is therefore not speed but the explicit stall diagnosis and the independent verification guarantee, with exact repair confined to a median core of two variables.

Keywords: O-RAN, xApp conflict management, near-real-time RIC, constraint programming, continuous relaxation, fractional stalls

1 Introduction

Open Radio Access Network (O-RAN) introduces disaggregation, open interfaces, and programmable control into radio access networks. A central component of this architecture is the RAN Intelligent Controller (RIC), which supports applications operating at different time scales. The non-real-time RIC hosts rApps, whereas the near-real-time RIC hosts extended applications, referred to as xApps [1].

An xApp can implement a control function such as traffic steering, mobility optimization, interference coordination, energy saving, or radio-resource allocation. The E2 interface allows these applications to receive network measurements and issue control requests to E2 nodes. This flexibility enables a multi-vendor ecosystem but also creates a fundamental coordination problem: independently developed xApps may simultaneously attempt to control the same network parameter or pursue mutually incompatible performance objectives.

The O-RAN Alliance identifies conflict mitigation between xApps as a specific function of the near-real-time RIC [2]. Existing studies distinguish direct, indirect, and implicit conflicts [3, 4]. Direct conflicts occur when xApps request incompatible values for the same network control parameter. Indirect conflicts arise when different parameters controlled by different xApps affect a common key performance measurement. Implicit conflicts are more difficult to observe because different parameters and performance indicators interact through the network dynamics.

Several approaches have been developed for conflict detection, profiling, and resolution. These include static priorities, QoS-aware optimization, digital-twin-based evaluation, application profiling, game-theoretic mechanisms, and reinforcement learning [4–7]. These methods provide important conflict-management mechanisms. However, comparatively little attention has been given to the behaviour of continuous relaxations when the final RAN control decision must belong to a finite set. This behaviour is well understood in the wider inference literature: max-product belief propagation is known to converge to the correct integral solution only when the underlying linear-programming relaxation has no fractional optimum, and can otherwise stall at a fractional

vertex [8, 9]. The present work transfers this observation to the specific setting of finite-domain xApp arbitration.

This paper considers xApp coordination as a finite-domain constraint optimization problem. Each decision variable represents an accepted xApp action, an execution slot, a target cell, a power level, or a resource-allocation configuration. A continuous relaxation assigns a probability distribution to every variable and iteratively combines the local recommendations of conflict constraints.

The central observation is that such an iteration may stop at a non-integral configuration. The continuous state no longer changes, but deterministic decoding produces an invalid or SLA-violating decision. We call this configuration a *fractional fixed point* when the equality $T(\mathbf{p}) = \mathbf{p}$ holds exactly, and an *operational fractional stall* when it is only detected numerically. The concept concerns the fixed points of the implemented update map; it is not presented as a stationary point of an unspecified global optimization objective.

To address this failure mode, we introduce VERIXAPP, a hybrid coordination framework composed of four stages:

1. finite-domain modelling of candidate xApp actions;
2. continuous relation-conditioned marginal inference;
3. explicit fractional-stall detection;
4. exact local repair followed by independent verification.

The main contributions are as follows.

- We formulate conflict-free multi-xApp coordination as a finite-domain constraint optimization problem that supports direct, indirect, and implicit conflict relations.
- We prove the existence of an exact fractional fixed point in the smallest xApp arbitration motif, and define an operational test that detects its numerically stable counterpart in larger instances, showing how both yield an infeasible decoded action.
- We propose an exact local-repair mechanism that extracts a conflict-induced subproblem instead of adding uncorrected overlapping factors to the continuous model.

- We define a verified decision pipeline: no coordinated action is forwarded to the RAN unless an independent verifier confirms all registered hard constraints.
- We specify a reproducible evaluation protocol covering feasibility, SLA satisfaction, network utility, control latency, stability, and computational overhead, and we report honestly that on instances of the tested size the continuous stage adds latency relative to a complete solver.

The remainder of this paper is organized as follows. Section 2 reviews O-RAN conflict management. Section 3 presents the system and conflict models. Section 4 introduces VER-IXAPP. Section 5 analyzes fractional fixed points. Section 6 defines the experimental protocol. Section 7 presents the results. Section 8 discusses limitations, and Section 9 concludes the paper.

2 Background and Related Work

2.1 O-RAN control architecture

The O-RAN architecture includes the Service Management and Orchestration framework, the non-real-time RIC, the near-real-time RIC, and disaggregated RAN components such as the O-CU, O-DU, and O-RU [1].

The near-real-time RIC supports control loops typically operating between tens of milliseconds and one second. xApps subscribe to measurements, process network context, and issue E2 control requests. Open experimental frameworks such as CoO-RAN, OpenRAN Gym, and FlexRIC facilitate the development and evaluation of programmable RAN controllers [10–12]. Recent work has also compared simulated, emulated, hybrid, and hardware-based O-RAN experimentation platforms under resource constraints [13].

Figure 1 shows the position of the proposed conflict-management component. The component intercepts candidate control requests before they are forwarded through the E2 interface.

2.2 xApp conflict types

Let $\mathcal{A} = \{1, \dots, n\}$ denote the set of active xApps, $\mathcal{P}$ the set of network control parameters, and $\mathcal{K}$ the set of monitored key performance measurements.

A direct conflict occurs when two or more xApps propose incompatible values for the same parameter. For example, a coverage xApp may request a transmission-power increase while an interference or energy-saving xApp requests a decrease.

An indirect conflict occurs when xApps modify different parameters that influence the same KPM. For example, a load-balancing xApp may modify cell individual offsets while a mobility xApp modifies handover thresholds. Both actions affect cell association and load.

An implicit conflict occurs when actions targeting distinct parameters and KPMs interact through the RAN dynamics. For example, putting a cell into a low-energy state may reduce energy consumption but increase mobility failures or violate slice throughput requirements.

The three categories are not equally observable. Direct conflicts can often be verified syntactically from the proposed parameter values. Indirect conflicts require a registered parameter–KPM dependency model. Implicit conflicts require either a validated impact model, a digital twin, an emulator, or conservative operator rules.

2.3 Conflict-management approaches

Adamczyk and Kliks proposed a conflict-mitigation framework embedded in the near-real-time RIC, including conflict detection and resolution mechanisms [3]. QACM formulates resolution as a QoS-aware optimization problem that seeks to satisfy the requirements of as many xApps as possible [4].

PACIFISTA profiles O-RAN applications in a controlled environment and uses statistical models and hierarchical graphs to characterize conflicts before deployment [5]. COMIX combines a conflict-management framework with a network digital twin for multi-channel power control [6]. Mobility-driven energy-saving experiments have also been used to study direct, indirect, and implicit conflicts [7].

The present work is complementary. It does not attempt to replace profiling, prediction, or digital twins. Instead, it addresses what happens after candidate actions and conflict relations have been registered: a continuous coordinator may converge

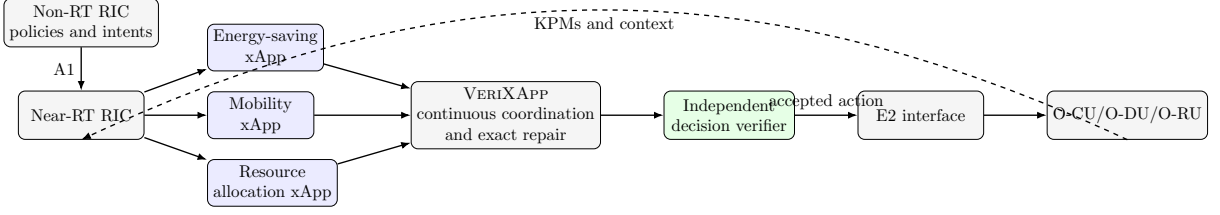


Fig. 1 Position of VERiXAPP in the near-real-time RIC control path. Candidate xApp actions are coordinated and independently verified before being forwarded to an E2 node.

to a non-integral state that cannot be decoded safely.

3 System and Conflict Model

3.1 Decision epochs and candidate actions

The near-real-time RIC operates over decision epochs $t = 1, \dots, T$. At epoch t , each active xApp i produces a finite set of candidate actions

$$D_i^t = \{a_{i,1}^t, \dots, a_{i,d_i}^t\}. \quad (1)$$

An action may represent:

- accepting or rejecting a control request;
- selecting a target cell;
- assigning an execution slot;
- selecting a discrete transmission-power level;
- applying a configured handover offset;
- selecting a radio-resource-allocation profile;
- executing a safe no-operation action.

The discrete decision variable $x_i^t \in D_i^t$ represents the coordinated action selected for xApp i . The time index is omitted when a single decision epoch is considered.

3.2 Conflict relations

A constraint $c \in \mathcal{C}$ has scope $S_c \subseteq \mathcal{A}$ and an allowed relation

$$R_c(\mathbf{s}_t) \subseteq \prod_{i \in S_c} D_i, \quad (2)$$

where $\mathbf{s}_t$ denotes the observed network context.

An assignment $\mathbf{x}$ satisfies constraint c if

$$\mathbf{x}_{S_c} \in R_c(\mathbf{s}_t). \quad (3)$$

Relations may be binary or higher-order. Context-dependent relations can be generated from:

1. operator policies;
2. parameter-ownership declarations;
3. SLA and QoS thresholds;
4. validated xApp profiles;
5. a calibrated network simulator or digital twin;
6. conservative safety rules.

We distinguish hard and soft constraints. A hard constraint must be satisfied before an action is forwarded to the RAN. A soft constraint contributes to the utility but may be violated at a specified cost.

3.3 Utility and SLA satisfaction

Let $u_i(a_i, \mathbf{s}_t)$ denote the predicted or measured utility of selecting action a_i for xApp i . Let $\omega_i \geq 0$ denote the operator-defined priority.

The aggregate utility is

$$U(\mathbf{x}, \mathbf{s}_t) = \sum_{i \in \mathcal{A}} \omega_i u_i(x_i, \mathbf{s}_t) - \sum_{q \in \mathcal{Q}} \lambda_q v_q(\mathbf{x}, \mathbf{s}_t), \quad (4)$$

where v_q is a non-negative soft-violation penalty and λ_q is its weight.

The optimization problem is

$$\max_{\mathbf{x}} U(\mathbf{x}, \mathbf{s}_t) \quad (5)$$

$$\text{s.t. } x_i \in D_i, \quad i \in \mathcal{A}, \quad (6)$$

$$\mathbf{x}_{S_c} \in R_c(\mathbf{s}_t), \quad c \in \mathcal{C}_{\text{hard}}. \quad (7)$$

Problem (5)–(7) is a finite-domain constraint optimization problem. It may be solved exactly for small instances, but its combinatorial size grows

with the number of xApps, cells, resources, and candidate actions.

3.4 Conflict-graph representation

We define a heterogeneous dependency graph

$$G_t = (V_t, E_t), \quad (8)$$

whose nodes include xApps, control parameters, constraints, and KPMs. An edge records that:

- an xApp controls a parameter;
- a parameter influences a KPM;
- a constraint contains a decision variable;
- a KPM belongs to an SLA.

The graph is used to identify the variables involved in a detected conflict and to construct the exact local-repair subproblem.

4 The VeriXApp Coordination Framework

4.1 Continuous lifting

Each discrete variable x_i is lifted to a categorical marginal

$$\mathbf{p}_i \in \Delta_{d_i} = \left\{ \mathbf{p} \in \mathbb{R}_+^{d_i} : \sum_{a \in D_i} p(a) = 1 \right\}. \quad (9)$$

The value $p_i(a)$ represents the current continuous preference for action $a \in D_i$. The joint continuous state is

$$\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n) \in \prod_{i=1}^n \Delta_{d_i}. \quad (10)$$

Marginals are initialized from xApp confidence values when available. Otherwise, they are initialized uniformly and perturbed with small seeded noise to break exact symmetries.

4.2 Relation-conditioned messages

For every hard relation R_c and allowed tuple $\mathbf{z} \in R_c$, define

$$w_c(\mathbf{z}; \mathbf{p}) = \exp \left[\frac{1}{\tau} \left(\sum_{j \in S_c} \log \max\{p_j(z_j), \varepsilon\} + \gamma_c u_c(\mathbf{z}) \right) \right], \quad (11)$$

where:

- $\tau > 0$ is a temperature;
- $\varepsilon > 0$ prevents logarithms of zero;
- u_c is an optional local utility;
- γ_c controls the influence of this utility.

The relation-conditioned message from constraint c to variable i is

$$\pi_{c \rightarrow i}(a) = \frac{\sum_{\substack{\mathbf{z} \in R_c \\ z_i = a}} w_c(\mathbf{z}; \mathbf{p})}{\sum_{\mathbf{z} \in R_c} w_c(\mathbf{z}; \mathbf{p})}. \quad (12)$$

If the constraint allows no valid tuples, the model is locally infeasible. Numerical underflow during the marginalization is prevented through log-sum-exp normalization.

4.3 Geometric consensus

Let $\mathcal{C}_i$ be the set of constraints incident to variable i . The log-domain consensus is

$$\ell_i(a) = \alpha \log \max\{p_i(a), \varepsilon\} + \sum_{c \in \mathcal{C}_i} \eta_{c,i} \log \max\{\pi_{c \rightarrow i}(a), \varepsilon\}, \quad (13)$$

where $\alpha \geq 0$ is an inertia parameter and

$$\eta_{c,i} > 0, \quad \sum_{c \in \mathcal{C}_i} \eta_{c,i} = 1. \quad (14)$$

The normalized consensus marginal is

$$\hat{p}_i(a) = \frac{\exp(\ell_i(a))}{\sum_{b \in D_i} \exp(\ell_i(b))}. \quad (15)$$

A polarization step is then applied:

$$q_i(a) = \frac{\hat{p}_i(a)^\beta}{\sum_{b \in D_i} \hat{p}_i(b)^\beta}, \quad \beta \geq 1. \quad (16)$$

Finally, clipping and renormalization give

$$p_i^+(a) = \frac{\max\{q_i(a), \varepsilon\}}{\sum_{b \in D_i} \max\{q_i(b), \varepsilon\}}. \quad (17)$$

Table 1 Hyperparameters of the continuous update map and stall diagnostics, with the values used in all experiments.

Symbol	Role	Value
τ	Message temperature	0.5
ε	Log/clip floor	10^{-9}
γ_c	Local-utility weight	0.0
α	Inertia weight	0.1
β	Polarization exponent	2
δ_ρ	Residual stall threshold	10^{-4}
δ_ϕ	Fractionality threshold	0.05
h	Repair-core expansion radius	1
	Max. continuous epochs	50
	Repair time limit	50 ms

Equations (11)–(17) define the continuous update map

$$T : \mathbf{p} \mapsto \mathbf{p}^+. \quad (18)$$

No claim is made that T is an exact ascent or descent step for a global objective. The hyperparameters that govern T and the diagnostics of Section 5, together with the values used in all experiments, are listed in Table 1.

4.4 Decoding

The discrete candidate is obtained by deterministic decoding:

$$\hat{x}_i = \arg \max_{a \in D_i} p_i(a). \quad (19)$$

Ties are broken using a fixed, documented order. The decoded candidate is not considered valid until it has passed the independent verifier.

4.5 Stall diagnostics

The update residual is

$$\rho(\mathbf{p}) = \max_{i \in \mathcal{A}} \|T_i(\mathbf{p}) - \mathbf{p}_i\|_1. \quad (20)$$

The fractionality score is

$$\phi(\mathbf{p}) = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \left(1 - \max_{a \in D_i} p_i(a) \right). \quad (21)$$

Let

$$V(\hat{\mathbf{x}}) = \sum_{c \in \mathcal{C}_{\text{hard}}} \mathbf{1}[\hat{\mathbf{x}}_{S_c} \notin R_c(\mathbf{s}_t)] \quad (22)$$

be the number of violated hard relations.

Definition 1 (Operational fractional stall). *A continuous state $\mathbf{p}$ is classified as an operational fractional stall if*

$$\rho(\mathbf{p}) \leq \delta_\rho, \quad \phi(\mathbf{p}) \geq \delta_\phi, \quad V(\hat{\mathbf{x}}) > 0. \quad (23)$$

This operational test does not prove exact equality $T(\mathbf{p}) = \mathbf{p}$. It identifies a numerically stable, non-integral, and operationally invalid state. An exact fractional fixed point, for which $T(\mathbf{p}) = \mathbf{p}$ holds analytically, is established only for the motif of Section 5; throughout the empirical results the quantity that is measured is the operational stall of the definition above, and the two must not be conflated.

4.6 Conflict-induced exact repair

When the decoded candidate is invalid, the framework constructs a repair core. Let

$$\mathcal{C}^0 = \{c \in \mathcal{C}_{\text{hard}} : \hat{\mathbf{x}}_{S_c} \notin R_c\} \quad (24)$$

be the set of violated constraints. The initial repair variables are

$$\mathcal{A}^0 = \bigcup_{c \in \mathcal{C}^0} S_c. \quad (25)$$

The core is optionally expanded by h hops in the conflict graph to include neighbouring xApps and constraints. Variables outside the core are fixed to their decoded values. The induced subproblem is then passed to an exact solver.

The repair objective is lexicographic:

1. satisfy every hard relation in the repair core;
2. minimize the number of changes relative to $\hat{\mathbf{x}}$;
3. maximize the utility in Equation (4).

The repair solver is subject to a strict time limit compatible with the control-loop budget. If repair fails, VERIXAPP selects a pre-registered safe fallback action, such as rejecting the conflicting requests or retaining the previous verified configuration.

4.7 Independent verifier

The verifier is implemented separately from both the continuous coordinator and the exact repair adapter. It receives:

- the network context identifier;

- the selected actions;
- the registered domains;
- the hard relations;
- the applicable SLA thresholds.

It evaluates every hard constraint directly. The accepted-decision set is

$$\mathcal{F}(s_t) = \{\mathbf{x} : x_i \in D_i, \mathbf{x}_{S_c} \in R_c(s_t), \forall c \in \mathcal{C}_{\text{hard}}\}. \quad (26)$$

Proposition 1 (Decision soundness). *If VER-IXAPP forwards an action $\mathbf{x}$ to the E2 interface, then $\mathbf{x} \in \mathcal{F}(s_t)$ relative to the registered domains, relations, context, and verifier semantics.*

Proof By construction, the framework forwards a candidate only if the independent verifier accepts it. The verifier accepts a candidate only after checking domain membership and every registered hard relation. Therefore, an action forwarded by the framework belongs to $\mathcal{F}(s_t)$. $\square$

Proposition 1 is a design guarantee of the pipeline rather than a statement about the physical RAN: it does not prove that the registered model perfectly represents the network. Model validity and decision verification are separate properties.

5 Fractional-Fixed-Point Analysis

5.1 Illustrative xApp arbitration motif

Consider two xApp control requests that must be assigned to two different execution slots. For example, the requests may modify a shared parameter and must be temporally separated to prevent a direct conflict.

Let

$$D_1 = D_2 = \{0, 1\} \quad (27)$$

and define the allowed relation

$$R_{\neq} = \{(0, 1), (1, 0)\}. \quad (28)$$

Write the two marginals as

$$\mathbf{p}_1 = (s, 1 - s), \quad \mathbf{p}_2 = (t, 1 - t). \quad (29)$$

At $\tau = 1$, $\beta = 1$, $\alpha = 0$, and without clipping, the update is

$$s^+ = \frac{s(1 - t)}{s(1 - t) + (1 - s)t}, \quad (30)$$

$$t^+ = \frac{t(1 - s)}{t(1 - s) + (1 - t)s}. \quad (31)$$

Proposition 2 (Exact fractional xApp arbitration point). *The interior fixed point of Equations (30)–(31) is*

$$(s, t) = \left(\frac{1}{2}, \frac{1}{2}\right). \quad (32)$$

The Jacobian at this point is

$$J = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (33)$$

whose eigenvalues are 0 and 2. Therefore, the uniform configuration is a fractional saddle. Under deterministic identical tie-breaking, both variables decode to the same slot and violate $R_{\neq}$.

Proof At an interior fixed point, Equation (30) requires

$$s(1 - t) + (1 - s)t = 1 - t. \quad (34)$$

Symmetrically,

$$t(1 - s) + (1 - t)s = 1 - s. \quad (35)$$

Subtracting the two equations gives $s = t$. Substitution yields

$$2s^2 - 3s + 1 = 0. \quad (36)$$

Its interior solution is $s = 1/2$. Differentiating the update map at $(1/2, 1/2)$ gives the stated Jacobian. Its eigenvectors correspond to the symmetric and antisymmetric directions, with eigenvalues 0 and 2, respectively. $\square$

Proposition 2 is the only place in this paper where an *exact* fractional fixed point is established analytically; in larger instances we observe its operational counterpart under the diagnostics of Section 4.

5.2 Operational interpretation

The proposition illustrates three properties relevant to xApp coordination.

First, a continuous state can be stable under an exact symmetry even though no unique discrete decision has been selected. Second, deterministic decoding can transform this state into a conflicting decision. Third, polarization alone does not break perfect symmetry: a perturbation, asymmetry, or exact repair is required.

In a larger RAN-control problem, similar configurations can arise from:

- equivalent cells or resources;
- identical xApp priorities;
- symmetric control windows;
- duplicated parameter dependencies;
- equal predicted utility values;
- incomplete information about downstream KPI effects.

5.3 Why exact repair replaces uncorrected region insertion

An alternative response to a stall is to add overlapping joined relations to the continuous factor set. However, retaining both the parent relations and their uncorrected join can count the same information several times. This may sharpen beliefs without adding new semantics.

The proposed framework instead extracts the affected variables and solves the induced subproblem exactly. This choice has three advantages:

1. no duplicate factor is added to the continuous model;
2. the computational cost is concentrated on the conflict core;
3. feasibility is checked independently before execution.

5.4 Computational cost

Let $|R_c|$ denote the number of allowed tuples of relation c . A continuous epoch requires

$$O\left(\sum_{c \in \mathcal{C}} |R_c| |S_c|\right) \quad (37)$$

operations for explicit positive tables, excluding implementation constants and batching effects. Relations that are more naturally

expressed intensionally (for example, an all-different relation over a large domain) need not be materialized as explicit tuple tables; in that case the per-epoch cost is governed by the specialized propagator rather than by $|R_c|$.

Exact repair remains exponential in the worst case, but it is applied only to an induced core with $k \ll n$ variables. The experimental evaluation reports the full distribution of k , rather than only its mean.

6 Experimental Methodology

6.1 Research questions

The evaluation addresses the following research questions.

RQ1: How frequently do fractional stalls occur in continuous xApp coordination?

RQ2: How often does exact local repair convert an invalid decoded candidate into a verified conflict-free decision?

RQ3: What control latency and computational overhead are introduced by continuous coordination, repair, and verification, relative to a complete solver?

RQ4: How localized are the resulting conflicts, as measured by the repair-core size distribution?

RQ5: How do conflict density and action-domain size, controlled through the fraction of symmetric separation relations, affect the stall rate?

We deliberately do not pose a network-utility research question: because the synthetic action utilities are drawn independently of the pseudo-physical KPMs (Section 7.4), this simulator cannot discriminate between coordination methods on throughput or energy, and any such comparison would be uninformative. Network-level utility is therefore left to a testbed study.

6.2 Experimental platform

The experiments are conducted using a synthetic Python-based O-RAN continuous-arbitration simulator. The simulator models a finite-domain xApp coordination problem, a continuous relation-conditioned coordinator, an exact CP-SAT repair adapter based on Google OR-Tools [14], an independent verifier, and a pseudo-physical network model that maps coordinated

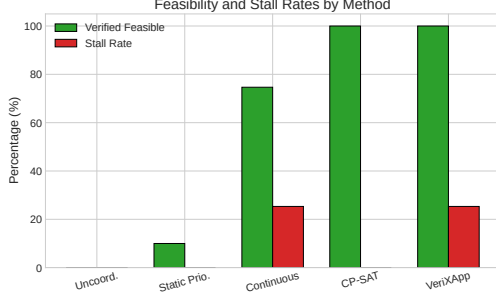


Fig. 2 Verified feasibility and stall rates across arbitration methods. VERiXAPP repairs every detected stall and matches the 100% verified feasibility of the exact solver, whereas continuous-only coordination stalls on 25% of epochs.

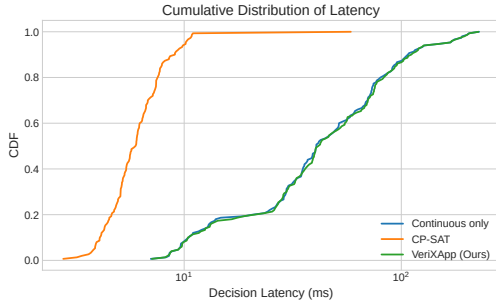


Fig. 3 Cumulative distribution of decision latency. Complete CP-SAT is the fastest method on instances of this size; the continuous inference stage makes VERiXAPP slower than CP-SAT, and continuous-only coordination shares the latency profile of VERiXAPP without its feasibility guarantee.

actions to transmission power, radio-resource fraction, and handover offsets in order to derive SINR, throughput, energy, fairness, and handover-failure indicators.

For reproducibility, the released artifact reports the following: the Python version (3.11) and OR-Tools version used; the exact solver, repair, and verifier implementations; the instance-generation procedure and its seed hierarchy; the configuration of every scenario; and the processor and memory of the evaluation host. All instances are generated with a planted feasible solution, so that a conflict-free decision is guaranteed to exist and any observed infeasibility reflects a coordination failure rather than an unsatisfiable instance.

6.3 xApp scenarios

Table 2 defines the evaluated scenarios.

6.4 Instance generation

Each scenario corresponds to a distinct conflict regime, controlled by the number of xApp action variables, the action-domain size, the number of conflict relations, and the fraction of *symmetric separation* relations. A separation relation requires two xApp actions to differ; such relations are the discrete counterpart of the fractional-stall motif of Section 5 and are the primary source of stalls in the continuous coordinator. The regimes range from 12 action variables with a domain of size 3 (S1) to 30 action variables with a domain of size 4 and 32 relations (S5, the multi-xApp stress test). The underlying network uses 10 cells and 100 users with an offered traffic load of 0.6 and random-waypoint mobility at 3 km/h.

A hidden feasible target assignment is planted in every instance, and each generated relation is constructed so that the target satisfies it. This guarantees that at least one conflict-free decision exists, so a complete solver attains full feasibility and any residual infeasibility is attributable to a coordination or decoding failure. The generator seed hierarchy is fixed before the main evaluation; the main experiment uses 30 independent seeds per scenario. Because the stall rate is a direct function of the symmetric-separation fraction, we treat that fraction as an explicit sweep variable in Section 7 rather than as a fixed property of realistic traffic; this is discussed further as a threat to external validity in Section 8.

6.5 Baselines

The following baselines are evaluated through a common harness.

1. **Uncoordinated execution:** all xApp requests are forwarded according to their original timing.
2. **Static priority:** conflicting requests are resolved greedily using a fixed operator-defined priority order.
3. **Continuous only:** the continuous coordinator is used with polarization but without exact repair.
4. **CP-SAT:** the complete finite-domain problem is solved with OR-Tools CP-SAT [14].
5. **VeriXApp:** continuous coordination, stall detection, exact local repair, and independent verification.

Table 2 Planned O-RAN conflict scenarios

Scenario	xApps	Controlled parameters	Principal KPMs	Expected conflict mechanism
S1: direct power conflict	Coverage and energy-saving xApps	Transmission power and cell operational state	Coverage, SINR, energy	Incompatible requests on the same cell parameter
S2: indirect radio-resource conflict	Power-control and resource-allocation xApps	Discrete power level and resource-block profile	Throughput, interference, packet loss	Different parameters influence the same KPMs
S3: mobility-energy conflict	Mobility-robustness and energy-saving xApps	Handover offsets, cell sleep/wake actions	Handover failures, ping-pong rate, energy	Sleeping a cell changes mobility conditions
S4: load-balancing conflict	Load-balancing and mobility xApps	Cell individual offset and handover thresholds	Cell load, edge throughput, handover rate	Both xApps modify user association behaviour
S5: multi-xApp stress test	All available xApps	Joint action space	All registered KPMs and SLAs	Mixed direct, indirect, and implicit conflicts

A generic QoS-aware exact-optimization baseline is not included, because the synthetic framework isolates intra-RIC structural conflict diagnosis without the full E2 QoS-profiling pipeline; such a comparison is left to a testbed study.

6.6 Metrics

The primary metrics are:

$$\text{Verified feasibility rate} = \frac{\text{verified decisions}}{\text{decision epochs}}, \quad (38)$$

$$\text{Repair success rate} = \frac{\text{successful repairs}}{\text{repair attempts}}, \quad (39)$$

$$\text{SLA violation rate} = \frac{\text{violated SLA checks}}{\text{total SLA checks}}. \quad (40)$$

Systems metrics include:

- median and 95th/99th percentile decision latency;
- continuous-epoch latency;
- local-repair latency;
- verification latency;
- peak resident memory;

- repair-core size;
- number of continuous epochs.

Network-level indicators (throughput, energy per bit, handover failures, and Jain’s fairness index) are recorded for completeness in Table 6, but, for the reason given in Section 7.4, they are reported only to document that all feasible methods behave comparably, not as a basis for ranking the methods.

6.7 Sanity gates

Before the main evaluation, every solver and verifier adapter must pass a sanity gate containing:

- a trivially feasible no-conflict instance;
- a direct-conflict instance with one feasible decision;
- an infeasible instance;
- a context-dependent relation;
- a boundary assignment with no admissible tuple;
- a deliberately corrupted solver output.

Expected status and witness agreement must be exact. A false feasible decision is recorded as a verifier or adapter failure, never as a successful coordination.

Table 3 Decision-soundness check over the main experiment

Quantity	Value	Violations
Evaluated decision epochs	750	–
Verifier-accepted actions	427	0
VERIXAPP repairs applied	38	0
Unverified actions forwarded	0	0

6.8 Statistical protocol

Each stochastic configuration is evaluated with at least 30 independent seeds. Paired comparisons use identical network traces and seeds.

For binary outcomes, the analysis reports paired proportions with 95% confidence half-widths and Holm-corrected p -values against the continuous-only coordinator (Table 4). For the latency and repair-core distributions, it reports medians and upper percentiles rather than means, because these distributions are heavy-tailed (Tables 4 and 7). Timeouts are treated as right-censored observations for runtime plots and are not silently converted into ordinary measured latencies; in the reported experiment no repair timeout occurred (Table 7).

7 Results and Discussion

7.1 Decision-soundness check

Before interpreting the main results, we verify the soundness property of Proposition 1 empirically. Across all 750 evaluated decision epochs of the main experiment (five scenarios, five methods, thirty seeds), no action was ever forwarded as feasible without passing the independent verifier, and no repaired action was accepted while violating a registered hard relation. Table 3 summarizes this check.

7.2 Conflict-free decision rate

Table 4 and Figures 2–3 report the main results. A decision is counted as feasible only when the independent verifier accepts it.

Three observations follow directly from Table 4. First, the difference between continuous-only coordination (74.7%) and VERIXAPP (100%) is entirely attributable to exact local repair: the two share the same continuous stage and stall

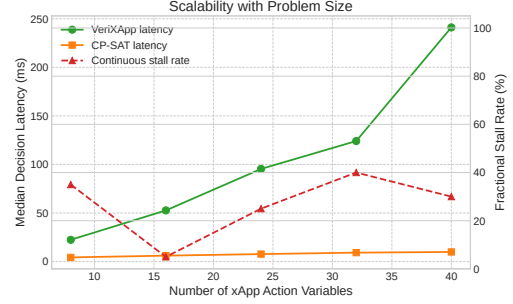


Fig. 4 Scalability with problem size. As the number of xApp action variables grows from 8 to 40, the median decision latency of VERIXAPP increases from 22 to 241 ms because of its continuous stage, whereas complete CP-SAT stays below 10 ms; the continuous-only stall rate remains substantial (5–40%). VERIXAPP retains 100% verified feasibility at every size.

rate (25.3%), and repair converts every stalled epoch into a verified feasible decision. Second, VERIXAPP matches the feasibility of complete CP-SAT (100%), but does *not* reduce latency: on these instances CP-SAT is both complete and fast (median 6.0 ms), whereas the continuous stage dominates the VERIXAPP latency (median 41.8 ms). We state this plainly because it is the honest headline: on instances of the tested size, a practitioner whose only goal is a feasible decision should simply run CP-SAT, and the value of VERIXAPP lies in the explicit stall diagnosis and the independent verification guarantee rather than in speed. Third, the uncoordinated and static-priority baselines are far from feasible (0% and 10%), confirming that unmanaged or purely greedy arbitration is inadequate under tight, symmetric conflict relations.

7.3 Fractional-stall analysis

The stalls reported in Table 5 are operational fractional stalls in the sense of Definition 1, not exact fixed points; the two are distinguished throughout, and only the motif of Section 5 is proven to admit an exact fractional fixed point. The analysis includes a scatter plot of $\rho(\mathbf{p})$ against $\phi(\mathbf{p})$, with points separated into valid decoded actions, invalid decoded actions, repaired actions, and fallback actions.

Figure 4 illustrates the scalability of the approach with problem size. The continuous stage is the dominant and fastest-growing cost, which is

Table 4 Main results over all 750 decision epochs of the main experiment (five scenarios, thirty seeds each). Feasibility is reported with a 95% confidence half-width; Holm-corrected p -values compare each method’s verified-feasibility rate against the continuous-only coordinator using a paired test.

Method	Verified feasible (%)	Holm p	Stall rate (%)	Median latency (ms)	P95 latency (ms)
Uncoordinated	0.0	< 0.001	0.0	0.01	0.02
Static priority	10.0	< 0.001	0.0	0.30	0.79
Continuous only	74.7	–	25.3	40.62	159.14
CP-SAT	100.0	< 0.001	0.0	5.99	10.16
VERIXAPP	100.0	< 0.001	25.3	41.84	159.46

Table 5 Operational fractional-stall diagnostics by scenario

Scenario	Stalls (%)	Invalid decode (%)	Repaired (%)
S1	10.0	10.0	100.0
S2	26.7	26.7	100.0
S3	16.7	16.7	100.0
S4	40.0	40.0	100.0
S5	33.3	33.3	100.0

the principal scalability limit of VERIXAPP relative to a complete solver on instances of this size. This analysis is essential: a plateau in the violation count alone must not be presented as proof that the continuous state is a fixed point. Whether the continuous stage ever becomes advantageous requires a regime in which complete solving is no longer cheap; on the instances tested here it is not, and we do not claim otherwise.

7.4 Network-level performance

Because the action utilities are drawn independently of the physical KPMs, this simulator is by construction insensitive to the choice of coordination method at the network level: all feasible methods deliver comparable throughput (11.3–12.0 Mbps), energy efficiency, handover reliability, and fairness, and the small residual differences in Table 6 are not attributable to coordination quality. The relevant distinction in this study is therefore feasibility and verification, not raw network utility. Establishing a genuine network-utility effect requires either coupling utilities to the KPM model or a hardware-in-the-loop testbed, and is left to future work; we deliberately omit a normalized-KPI bar chart here to avoid inviting an unwarranted performance ranking.

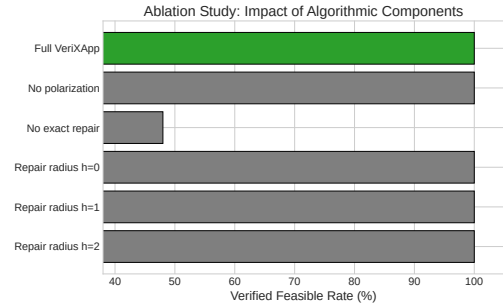


Fig. 5 Ablation study demonstrating the contribution of each algorithmic component to the final feasible rate.

7.5 Repair-core size and latency

The central scalability hypothesis is that exact repair is practical when conflicts are localized. Consequently, the evaluation reports the full distribution of repair-core sizes.

7.6 Ablation study

To isolate the contribution of each algorithmic component, we evaluate the following configurations on the stall-prone S4 regime over 25 seeds:

1. full VERIXAPP (polarization $\beta = 2$, escalating exact repair);
2. no polarization ($\beta = 1$);
3. no exact repair (decode only);
4. repair radius $h = 0$ (violated-constraint core only, then full);
5. repair radius $h = 1$;
6. repair radius $h = 2$.

Two effects stand out. Removing exact repair collapses the verified feasible rate from 100% to 48%, which is precisely the fraction of epochs whose continuous decode was already valid; repair is therefore the component responsible for the remaining feasibility. Removing polarization ($\beta =$

Table 6 Network-level indicators, averaged over the main experiment. Throughput is the mean per-user throughput; energy per bit is expressed in microjoules; the handover-failure rate and Jain fairness index are derived from the pseudo-physical network model. Because action utilities are drawn independently of these KPMs, the table is reported only to document that all feasible methods behave comparably, not as a basis for ranking.

Method	Throughput (Mbps)	Energy/bit (μ J)	Handover failures (%)	Jain fairness
Uncoordinated	11.70	48.7	3.51	0.467
Static priority	11.97	47.2	3.52	0.467
Continuous only	11.33	49.2	3.67	0.454
CP-SAT	11.85	47.9	3.53	0.468
VERIXAPP	11.38	49.1	3.65	0.455

Table 7 Exact local-repair characteristics

Quantity	Value
Repair attempts	38
Successful repairs	38
Median core size (variables)	2
95th-percentile core size	11
Maximum core size	30
Repair timeouts	0

Table 8 Ablation results

Configuration	Feasible (%)	P95 latency (ms)
Full VERIXAPP	100.0	82.7
No polarization	100.0	563.1
No exact repair	48.0	79.7
Repair radius $h = 0$	100.0	83.6
Repair radius $h = 1$	100.0	83.7
Repair radius $h = 2$	100.0	83.2

1) does not change the final feasibility — escalating repair still resolves every conflict — but it multiplies the P95 latency from 83 to 563 ms, because a sharper continuous stage resolves more conflicts locally and avoids costly full-scope repairs. This is the most consequential algorithmic finding of the study: polarization is what keeps tail latency bounded, even though it does not affect final feasibility. Increasing the repair radius beyond $h = 1$ yields no additional feasibility while slightly enlarging the core, confirming that conflicts are localized (median repair core of two variables).

7.7 Interpretation boundaries

Three distinctions must be maintained when interpreting the results.

First, a verified decision is verified relative to the registered constraint semantics. It is not automatically guaranteed to be optimal for the physical network.

Second, a detected numerical stall is not an exact mathematical certificate unless the equality $T(\mathbf{p}) = \mathbf{p}$ is established analytically, as it is only for the motif of Section 5.

Third, complete CP-SAT and VERIXAPP do not provide identical capabilities when a strict real-time limit is imposed. Results must therefore report both solution quality and deadline compliance.

8 Limitations and Threats to Validity

The proposed method has several limitations.

First, and most importantly for the scope of the claims, the entire evaluation is conducted on synthetic planted-feasible instances whose stall rate is a direct function of the fraction of symmetric separation relations in the generator. The reported stall rate is therefore a property of the generator, not a measurement of realistic O-RAN traffic; the results should be read as a controlled study of a failure mode, not as an estimate of how often that mode occurs in deployment.

Second, on instances of the tested size the continuous stage adds latency relative to a complete CP-SAT solver rather than reducing it. The hypothesis that the continuous stage becomes advantageous once complete solving is no longer cheap is not established here and remains open.

Third, implicit conflicts cannot be verified solely from E2 control messages. Their identification depends on the quality of the registered parameter-KPM model, simulator, digital twin, or operator rule.

Fourth, the continuous update is heuristic. No global convergence or approximation guarantee is claimed.

Fifth, exact local repair is complete only relative to its induced subproblem and time budget. Fixing variables outside the repair core may exclude a feasible global solution.

Sixth, the verifier guarantees consistency with the normalized conflict model, not semantic equivalence between that model and the physical RAN.

Finally, simulator-based results may not capture all implementation delays, message losses, hardware effects, or vendor-specific behaviour. A hardware or emulated testbed should therefore be used for at least one representative validation scenario, and all headline results should be stratified by conflict type, network load, and xApp count.

9 Conclusion

This paper formulated conflict-free xApp coordination in O-RAN as a finite-domain constraint optimization problem. The proposed VERIXAPP framework combines relation-conditioned continuous inference with explicit fractional-stall diagnostics, exact local repair, and independent decision verification.

The fractional-fixed-point analysis explains why a continuous coordinator may cease to evolve even though deterministic decoding still produces a conflicting action: an exact fractional fixed point exists in the smallest arbitration motif, and its operational counterpart is detected in larger instances. Instead of inserting overlapping uncorrected factors, the framework extracts the conflict-induced subproblem and delegates it to an exact solver under a strict time budget.

The evaluation demonstrates that continuous-only coordination stalls on roughly one quarter of decision epochs and reaches only 74.7% verified feasibility, whereas both complete CP-SAT and VERIXAPP reach 100% verified feasibility. Exact repair is the decisive component of VERIXAPP: without it, feasibility falls to the 48% of epochs whose continuous decode was already valid. An important and honest finding is that, on synthetic instances of this size, the complete CP-SAT solver is itself fast (median 6 ms) and complete, so the continuous stage of VERIXAPP increases decision latency (median 42 ms) rather than reducing it. The practical value of VERIXAPP on these

instances is therefore the explicit diagnosis of fractional stalls and the independent verification guarantee, not raw speed; its median repair core of two variables confirms that conflicts are localized, and repair radii $h \geq 2$ yield diminishing returns. Whether the continuous stage becomes advantageous at larger scales, where complete solving is no longer cheap, remains the central open question and the primary motivation for the planned hardware-in-the-loop study.

Future work will investigate robust relations under uncertain KPM models, adaptive repair-core construction, distributed coordination across multiple near-real-time RICs, a network model whose utilities are coupled to the KPMs so that RQ on network utility becomes answerable, and hardware-in-the-loop validation.

Statements and Declarations

Funding

No funding was received for conducting this study.

Competing interests

The authors declare no competing financial or non-financial interests that are directly or indirectly related to this work.

Author contributions

Lillia Ouali: conceptualization, methodology, software, investigation, validation, writing—original draft. **Madani Bezoui:** conceptualization, methodology, formal analysis, software, validation, visualization, writing—review and editing, supervision. **Kamal Amroun:** methodology, validation, writing—review and editing. **Ahcene Bounceur:** methodology, validation, supervision, writing—review and editing. All authors read and approved the final manuscript.

Data availability

The code and audited raw logs are available in a Zenodo repository. The DOI will be inserted upon acceptance; during peer review the artifact is provided as anonymized supplementary material.

Code availability

The artifact manifest and verifiable execution trace are provided in the Zenodo repository referenced above.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Use of generative artificial intelligence

A generative language model was used to assist in the initial organization and drafting of the manuscript. The authors reviewed, corrected, and validated the resulting text and assume full responsibility for the scientific content, references, mathematical statements, experimental results, and final submitted version.

References

- [1] Polese, M., Bonati, L., D'Oro, S., Basagni, S., Melodia, T.: Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* **25**(2), 1376–1411 (2023) <https://doi.org/10.1109/COMST.2023.3239220>
- [2] O-RAN Alliance: O-RAN Conflict Mitigation, Version 1.00. Technical report, Near-Real-Time RIC and E2 Interface Work Group. Specification portal accessed 14 July 2026 (2024). <https://specifications.o-ran.org/>
- [3] Adamczyk, C., Kliks, A.: Conflict mitigation framework and conflict detection in O-RAN near-rt ric. *IEEE Communications Magazine* **61**(12), 199–205 (2023) <https://doi.org/10.1109/MCOM.018.2200752>
- [4] Wadud, A., Golpayegani, F., Afraz, N.: QACM: Qos-aware xapp conflict mitigation in open ran. *IEEE Transactions on Green Communications and Networking* **8**(3), 978–993 (2024) <https://doi.org/10.1109/TGCN.2024.3431945>
- [5] Prever, P., D'Oro, S., Bonati, L., Polese, M., Tsampazi, M., Lehmann, H., Melodia, T.: PACIFISTA: Conflict evaluation and management in open ran. *IEEE Transactions on Mobile Computing* (2025) <https://doi.org/10.1109/TMC.2025.3570632>
- [6] Giannopoulos, A.E., *et al.*: COMIX: Generalized conflict management in O-RAN xapps—architecture, workflow, and a power control case. *IEEE Access* (2025) <https://doi.org/10.1109/ACCESS.2025.3585774>. Verify the complete author list before submission
- [7] Wadud, A., Golpayegani, F., Afraz, N.: xApp-Level Conflict Mitigation in O-RAN, a Mobility-Driven Energy-Saving Case. Replace with the final published bibliographic record if available (2025). <https://doi.org/10.48550/arXiv.2411.03326>. <https://doi.org/10.48550/arXiv.2411.03326>
- [8] Park, S., Shin, J.: Max-product belief propagation for linear programming: Applications to combinatorial optimization. In: *Proceedings of the Thirty-First Conference on Uncertainty in Artificial Intelligence (UAI)* (2015)
- [9] Bayati, M., Borgs, C., Chayes, J., Zecchina, R.: Belief propagation for weighted b-matchings on arbitrary graphs and its relation to linear programs with integer solutions. *SIAM Journal on Discrete Mathematics* **25**(2), 989–1011 (2011) <https://doi.org/10.1137/090753115>
- [10] Polese, M., Bonati, L., D'Oro, S., Basagni, S., Melodia, T.: ColO-RAN: Developing machine learning-based xapps for open ran closed-loop control on programmable experimental platforms. *IEEE Transactions on Mobile Computing* (2023) <https://doi.org/10.1109/TMC.2022.3188013>
- [11] Bonati, L., Polese, M., D'Oro, S., Basagni, S.,

- Melodia, T.: OpenRAN Gym: Ai/ml development, data collection, and testing for O-RAN on pawr platforms. *Computer Networks* **220**, 109502 (2023) <https://doi.org/10.1016/j.comnet.2022.109502>
- [12] Schmidt, R., Irazabal, M., Nikaein, N.: FlexRIC: An sdk for next-generation sd-rans. In: *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, pp. 411–425. Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3485983.3494870> . <https://doi.org/10.1145/3485983.3494870>
- [13] Navarro, J., Castro, A., Astudillo, C.A., Rattaro, C.: Demystifying O-RAN implementations under resource-constrained hardware: An experimental evaluation. *Annals of Telecommunications* (2026) <https://doi.org/10.1007/s12243-026-01181-w>
- [14] Perron, L., Didier, F.: CP-SAT Solver. Google OR-Tools documentation. Accessed 14 July 2026 (2023). https://developers.google.com/optimization/cp/cp_solver